

StockTool — Scope, Specification & Implementation Plan

Version: 0.1 (Pre-development)

Date: April 2026

Author: Architecture & Planning Document

1. Executive Summary

StockTool is a desktop-grade stock charting and analysis application inspired by MetaStock. It fetches OHLCV price data for US, HK, and global markets, stores it locally in CSV format, and presents it through a rich interactive charting UI. The system is designed with a modular architecture that supports progressive enhancement — from a basic charting MVP up to AI-driven analysis and automated trading notification in future phases.

2. Project Goals

Primary goals

- Provide professional-grade charting with candle, line, and multi-timeframe support
- Allow flexible technical analysis via configurable indicators (RSI, MACD, STC, moving averages)
- Store all data locally for offline use and historical analysis
- Support user drawing tools (trend lines) and chart export to PNG
- Surface relevant news alongside price data

Secondary goals (phased)

- Batch-scheduled data downloading with cron/task scheduler support
 - AI-powered candlestick pattern and trend analysis
 - Integration with a broker trading platform
 - Discord-based trade notifications
-

3. Target Users

- Individual retail traders and investors (primary)
- Quantitative analysts building and testing strategies (secondary)

- Portfolio managers requiring multi-market coverage (secondary)
-

4. Scope

4.1 In Scope — Phase 1 (MVP)

- Data fetching from yfinance (US, HK, and 50+ global exchanges)
- Local CSV storage with configurable data directory
- Start-date selection for historical data download
- Candle and line chart display
- Timeframe resampling: Daily, Weekly, Monthly, Quarterly, Yearly
- Moving average overlays: SMA, EMA, WMA (configurable period)
- Sub-chart indicators: Volume, RSI (configurable period), MACD (configurable fast/slow/signal), STC (configurable)
- Trend line drawing tool
- PNG chart export to configurable output directory
- Related news panel fetching headlines for the selected ticker

4.2 In Scope — Phase 2 (Enhanced)

- Batch download scheduler (daily/weekly cron)
- Multiple chart layouts (split view, comparison charts)
- Watchlist management with persistent storage
- Alert system (price, RSI level triggers)
- Additional indicators: Bollinger Bands, ATR, OBV, Stochastic, Ichimoku
- Drawing tools: Fibonacci retracement, horizontal levels, channels, rectangles, text annotations

4.3 In Scope — Phase 3 (AI & Trading)

- AI candlestick pattern recognition (hammer, doji, engulfing, head & shoulders, etc.)
- Volume / price divergence detection
- AI trend bias scoring (bullish / bearish / neutral)
- Backtesting engine with P&L and drawdown reporting
- Broker API integration (Interactive Brokers via ib_insync, or Alpaca)

- Discord webhook for signal and trade notifications

4.4 Out of Scope

- Real-time streaming tick data (delayed polling only)
 - Options or derivatives pricing
 - Portfolio accounting / P&L tracking (Phase 3+ consideration)
 - Mobile client
-

5. Recommended Additional Features

The following features are recommended based on common professional charting needs:

Chart enhancements

- Multi-pane layout (up to 4 tickers side by side)
- Chart templates — save and restore indicator configurations
- Comparison overlay (plot two tickers on the same chart, % normalised)
- Dark / light theme toggle

Data enhancements

- Earnings date overlays on the price chart
- Dividend and split event markers
- Economic calendar overlay (CPI, Fed meetings, etc.)
- Data integrity checks: detect and flag gaps or anomalies in CSV files

Workflow enhancements

- Watchlist with colour-coded status (above/below MA, RSI overbought/oversold)
- Notes panel — attach freeform trade journal text per ticker per date
- Session log — record when data was last updated per symbol

Analysis enhancements

- Market breadth indicators (advance/decline, new highs/lows)
 - Relative strength ratio chart vs index (e.g. stock vs SPY)
 - Sector rotation heat map
-

6. Technical Architecture

6.1 Technology Stack

Layer	Recommended Technology	Rationale
Language	Python 3.11+	Rich ecosystem for finance, charting, and AI
UI framework	PyQt6 or PySide6	Native desktop, mature, good charting integration
Charting	pyqtgraph or mplfinance + Matplotlib	pyqtgraph is faster for interactive use; mplfinance is simpler for export
Data fetching	yfinance (primary), alpha_vantage (fallback)	yfinance covers US + HK + global; free tier sufficient for retail use
Indicator library	pandas-ta or TA-Lib	pandas-ta is pure Python, easier install; TA-Lib faster for large data
News	NewsAPI.org or feedparser (RSS)	NewsAPI has good ticker-to-article mapping; RSS is free fallback
Scheduler	APScheduler or system cron	APScheduler integrates in-process; cron is simpler for batch
Storage	CSV files (flat) + SQLite (metadata)	CSV for portability; SQLite for watchlists, settings, annotations
AI (Phase 3)	OpenAI API / local Ollama	Structured JSON output for pattern classification
Broker (Phase 3)	ib_insync (IBKR) or alpaca-trade-api	Both have Python SDKs
Notifications	discord.py webhook	Simple HTTP POST, no bot required for one-way alerts

6.2 Directory Structure

```
stocktool/
├─ main.py           # Entry point
├─ config.yaml       # User configuration (paths, API keys, defaults)
├─ requirements.txt
└─
```

```

└─ core/
  │ └─ data_manager.py      # Fetch, validate, resample OHLCV data
  │ └─ indicator_engine.py  # SMA, EMA, WMA, RSI, MACD, STC, etc.
  │ └─ news_fetcher.py      # News API + RSS parser
  │ └─ scheduler.py         # Batch download job management
  │
  └─ ui/
    │ └─ main_window.py     # Main application window
    │ └─ chart_panel.py     # Price chart (candle/line) + MA overlays
    │ └─ indicator_panel.py  # Sub-chart area (Volume, RSI, MACD, STC)
    │ └─ toolbar.py         # Symbol search, timeframe, date picker
    │ └─ news_sidebar.py    # News headlines panel
    │ └─ drawing_tools.py   # Trend line, Fibonacci, annotation engine
    │ └─ settings_dialog.py # User preferences dialog
    │
    └─ models/
      │ └─ symbol.py        # Symbol metadata model
      │ └─ candle.py        # OHLCV candle model
      │ └─ indicator.py     # Indicator result model
      │
      └─ storage/
        │ └─ csv_store.py   # Read/write CSV price files
        │ └─ db_store.py    # SQLite: watchlists, annotations, settings
        │
        └─ ai/              # Phase 3
          │ └─ pattern_detector.py
          │ └─ trend_scorer.py
          │ └─ backtester.py
          │
          └─ integrations/  # Phase 3
            │ └─ broker_bridge.py
            │ └─ discord_notifier.py
            │
            └─ data/        # Default local data directory
              │ └─ prices/
              │   │ └─ US/      # e.g. AAPL.csv, MSFT.csv
              │   │ └─ HK/      # e.g. 0700.HK.csv
              │   └─ exports/   # PNG chart exports

```

6.3 CSV File Format

Each ticker gets its own CSV file. The file is append-friendly and stores daily OHLCV data.

```
Date,Open,High,Low,Close,Volume,Adj_Close
2024-01-02,185.22,188.44,184.10,187.15,55823400,187.15
2024-01-03,186.10,187.80,182.01,184.92,44211000,184.92
```

Resampled timeframes (Weekly, Monthly, etc.) are computed in memory from the daily CSV — they are not stored separately.

7. Feature Specifications

7.1 Symbol Search & Market Selection

- Text input with autocomplete backed by a local ticker list (loaded from a bundled CSV of US + HK + common global symbols)
- Market dropdown: All / US / HK / UK / JP / AU / Custom
- On symbol selection: auto-load data from CSV if it exists; prompt to download if not

7.2 Date Range & Start Date Selection

- Start date picker (calendar widget)
- End date defaults to today; user can set a custom end date for historical comparison
- On "Download" action: fetch from yfinance from start date to today, append/merge with existing CSV, avoid duplicate rows

7.3 Chart Display

Timeframes: D (daily), W (weekly), M (monthly), Q (quarterly), Y (yearly)

Display modes:

- Candle: standard OHLC candle with configurable bull/bear colours
- Line: close-price line chart

Moving average overlays (configurable per overlay):

- SMA (Simple Moving Average) — period configurable, e.g. 20, 50, 200
- EMA (Exponential Moving Average)
- WMA (Weighted Moving Average)
- Up to 5 simultaneous MA overlays, each with independent colour and period

Interaction:

- Mouse wheel zoom (time axis)
- Click-drag pan
- Crosshair cursor with OHLCV tooltip
- Right-click context menu for drawing tools

7.4 Sub-chart Indicators

Each sub-chart occupies a resizable pane below the main chart. Up to 4 sub-charts can be shown simultaneously.

Volume

- Bar chart coloured by candle direction
- Optional MA of volume overlay

RSI (Relative Strength Index)

- Configurable period (default: 14)
- Configurable overbought/oversold lines (default: 70/30)
- Coloured fill between RSI and 50 line

MACD (Moving Average Convergence Divergence)

- Configurable fast period (default: 12), slow period (default: 26), signal period (default: 9)
- Displays: MACD line, Signal line, Histogram bars (coloured by direction)

STC (Schaff Trend Cycle)

- Configurable fast (default: 23), slow (default: 50), cycle period (default: 10)
- Overbought (75) and oversold (25) threshold lines

7.5 Drawing Tools

Accessed via toolbar or right-click menu:

- **Trend line:** Click two points, drag to extend; configurable colour and line style
- **Horizontal level:** Single click places a horizontal line at that price
- **Ray / extended line:** Trend line extended to the right edge
- **Text annotation:** Click to place a text label at any chart point

Drawing state is saved per ticker per timeframe in SQLite and reloaded on next view.

PNG export:

- "Export Chart" button or File → Export
- Saves the current visible chart (including all indicators, overlays, and drawings) to the configured export directory
- Filename format: `{TICKER}_{TIMEFRAME}_{DATE}.png`
- Configurable resolution (default: 1920×1080)

7.6 News Panel

- Displays latest 20 headlines relevant to the selected ticker
 - Data source: NewsAPI.org (requires free API key) with RSS fallback
 - Refresh on ticker change or manual refresh button
 - Click headline to open article in the system browser
 - Optional: sentiment badge (positive / neutral / negative) using simple keyword scoring
-

8. Configuration (config.yaml)

```
yaml
```



```
data:
  price_dir: "./data/prices"
  export_dir: "./data/exports"

download:
  default_start_date: "2010-01-01"
  provider: "yfinance"          # yfinance | alpha_vantage
  alpha_vantage_key: ""

news:
  provider: "newsapi"          # newsapi | rss
  newsapi_key: ""
  max_headlines: 20

chart:
  default_timeframe: "D"
  candle_bull_color: "#26a69a"
  candle_bear_color: "#ef5350"
  background_color: "#131722"
  ma_colors: ["#2196F3", "#FF9800", "#E91E63", "#9C27B0", "#00BCD4"]
  export_resolution: [1920, 1080]

indicators:
  rsi_period: 14
  rsi_overbought: 70
  rsi_oversold: 30
  macd_fast: 12
  macd_slow: 26
  macd_signal: 9
  stc_fast: 23
  stc_slow: 50
  stc_cycle: 10

scheduler:
  enabled: false
  cron: "0 18 * * 1-5"          # Weekdays at 6pm
  symbols_file: "./watchlist.txt"
```

9. Implementation Plan

Phase 1 — Core MVP (Estimated: 6-8 weeks solo, 3-4 weeks with team)

Week	Milestone	Deliverables
1	Project scaffold	Repo structure, config loader, virtual env, dependency install
1-2	Data layer	csv_store.py, data_manager.py, yfinance integration, resample logic
2-3	Indicator engine	SMA/EMA/WMA, RSI, MACD, STC via pandas-ta
3-4	Chart panel	pyqtgraph candle + line chart, MA overlays, zoom/pan, crosshair
4-5	Indicator panel	Volume, RSI, MACD, STC sub-charts with param UI
5	Drawing tools	Trend line, horizontal level, text annotation, SQLite persistence
6	News panel	NewsAPI integration, headline list, browser open
6-7	PNG export	Export current chart view to PNG
7-8	Settings, polish	config dialog, error handling, symbol search autocomplete

Phase 2 — Enhanced Charting (Estimated: 4-6 weeks)

Task	Notes
Batch scheduler	APScheduler, cron config, progress log
Watchlist manager	SQLite-backed, colour-coded status
Additional indicators	Bollinger Bands, ATR, OBV, Stochastic
Drawing tools extended	Fibonacci, channels, rectangles
Multiple chart layouts	Split view, chart comparison overlay
Alert system	Price/RSI trigger with desktop notification

Phase 3 — AI & Trading (Estimated: 6-10 weeks)

Task	Notes
Candlestick pattern AI	LLM or CV model classifying chart screenshots
Divergence detector	RSI/MACD divergence vs price, rule-based first

Task	Notes
Trend bias scorer	Combines pattern + indicator signals into a score
Backtester	Apply strategy rules to historical CSV data
Broker bridge	IBKR via <code>ib_insync</code> ; paper trading first
Discord notifier	Webhook with chart image attachment

10. Key Dependencies

```
# requirements.txt (Phase 1)
PyQt6>=6.6.0
pyqtgraph>=0.13.3
yfinance>=0.2.37
pandas>=2.1.0
pandas-ta>=0.3.14b0
numpy>=1.26.0
requests>=2.31.0
feedparser>=6.0.10
pyyaml>=6.0.1
Pillow>=10.2.0
APScheduler>=3.10.4    # Phase 2
ib_insync>=0.9.86      # Phase 3
discord.py>=2.3.2      # Phase 3
```

11. Data Provider Notes

yfinance

- Covers NYSE, NASDAQ, HKEX (suffix `.HK`), LSE (`.L`), TSX (`.TO`), ASX (`.AX`), and more
- Free, no API key required for historical daily data
- Rate-limited; use `yf.download()` with `threads=False` for batch to avoid bans
- Data quality: generally reliable for daily OHLCV; adjust for splits/dividends using `auto_adjust=True`

HK stocks

- Use ticker format (0700.HK) (4-digit code + (.HK))
- HKEX closes at 16:00 HKT; data appears on yfinance with UTC timestamps

Alpha Vantage (fallback)

- Free tier: 25 API requests/day, 5 per minute
- Better for intraday data in Phase 2+
- Requires free API key registration

12. Risk Register

Risk	Likelihood	Impact	Mitigation
yfinance API changes or rate limiting	Medium	High	Implement provider abstraction layer; add Alpha Vantage fallback
NewsAPI free tier limits (100 req/day)	High	Low	Cache results; use RSS fallback; batch news requests
Performance lag with large CSV datasets (10+ years, 100+ tickers)	Medium	Medium	Lazy-load data; cache resampled frames; use numpy for indicator math
pyqtgraph limitations for advanced drawing tools	Medium	Medium	Evaluate switching to a custom QGraphicsScene for drawing layer
Broker API connectivity (Phase 3)	Low	High	Paper trading mode first; comprehensive error handling and reconnect logic

13. Suggested Development Sequence for First Code Sprint

The recommended order for the first two weeks of development:

1. (config.py) — load config.yaml, expose settings as typed dataclass
2. (csv_store.py) — read/write OHLCV CSVs, merge/append logic, detect gaps
3. (data_manager.py) — wrap yfinance, call csv_store, resample D→W→M→Q→Y
4. (indicator_engine.py) — compute all indicators from a DataFrame, return named series

5. `chart_panel.py` — pyqtgraph PlotWidget, draw candles, overlay MAs
6. `indicator_panel.py` — Volume sub-chart first, then RSI, MACD, STC
7. `main_window.py` — wire panels together, add toolbar, test end-to-end

This sequence prioritises getting a working chart on screen quickly, which is the most motivating milestone and surfaces layout/performance issues early.

Document prepared for development planning purposes. All timelines are estimates and assume a single experienced Python developer.